Cloud Computing with Google DataStore

By Don Schlichting

Introduction

DataStore is a database web service offered by Google.com as part of their App Engine development stack. The App Engine can be used to build and host web applications. DataStore is a non-relational cloud database that can be used along with the App Engine to store any data needed by the application. This article will examine how the DataStore is accessed, populated, and queried.

What is Cloud Computing?

Wikipedia defines Cloud Computing as "a style of computing in which resources are provided as a service over the internet". For me personally, Cloud Computing means developing or managing a machine or service I do not have physical responsibility for and is located somewhere in the internet. I further break down Cloud Computing into two roles of activity, either managing an entire virtual asset (virtual machine or application), or just interacting with a specific service. Google DataStore is the later because we interact with the database through a service and are not responsible for any operating system maintenance functions.

What is the DataStore?

The DataStore physically lives on Google's servers. It is spread across multiple servers to provide redundancy and performance. This is one of the main benefits of developing on the App Engine; Google's scalability is leveraged. The DataStore was developed over Google's Big Table, which hosts many internal and external Google services.

The DataStore is a non-relational database. This means unlike traditional databases, we don't build normalized tables then JOIN them for results. Instead, the DataStore is optimized for Read speed. The JOIN command is not supported so we need to architect the database as we would for a high volume reporting database, meaning more columns and fewer tables.

Many common datatypes are supported in the DataStore, including Sting, Int, Float, and DateTime. In addition, there is Key (system assigned unique row id), Links (URL), phone number, and postal address types. The complete list of Types is located at this URL: http://code.google.com/appengine/docs/python/datastore/typesandpropertyclasses.html .

Development on the DataStore is done though Application Programming Interfaces (API). These can be accessed by either Python or JAVA. There is a Management Console, but unlike the consoles in Oracle or MS SQL, the DataStore console isn't really designed to manipulate data. This is done though your own programming instead. In addition, there isn't any concept of a stored procedure or view. All database code is maintained in your JAVA or Python application.

Getting Started

The examples in the article will use Python 2.6.2 which can be downloaded free of charge at http://www.python.org/download/ . In addition, you'll also need the Google App Engine SDK (Software Development Kit) which can be downloaded from http://code.google.com/appengine/downloads.html. To get started, create a login with the Google App Engine, located at http://code.google.com/appengine/. Low usage accounts are free of charge.

Once a login is created with the App Engine, it will ask you to create an Account. Think of an Account as a Database. All our Tables (called Models in the DataStore), and data (Entities) will be saved inside an Application (Database).

Hello World

To verify your development environment is setup correctly, we'll create a very small test application. On your c:\ drive, create a helloworld directory. Next create a file called app.yaml . The YAML is a runtime configuration file. Enter the following text into the YAML file:

```
application: sqlbold
version: 1
runtime: python
api_version: 1

handlers:
- url: /hello.*
  script: hello.py
```

Change the first line "application" to your database name. The "handlers" section maps URLs to Python pages. In this example, any URL starting with "hello" will map to file we're going to created called "hello.py".

Next, create a file called "hello.py" and place it in the "helloworld" directory. Enter the following code into this file:

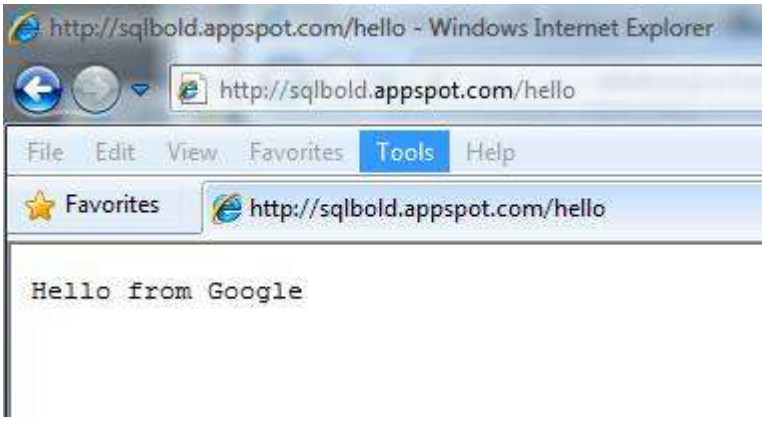
```
print 'Content-Type: text/plain'
print ''
print 'Hello from Google'
```

It doesn't mater what tool creates these files, notepad, the IDLE Python editor, or some other text application.

The next step is to push our application up to the Google server. Open a DOS or Command prompt and execute "appcfg.py update c:\helloworld". You will be prompted for your Google login. Below is the output from posting the application.

```
C:\Users\dons>appcfg.py update c:\helloworld
C:\Program Files\Google\google_appengine\appcfg.py:40: D
ha module is deprecated; use the hashlib module instead
  DIR_PATH,
Scanning files on local disk.
Initiating update.
Email: dons@pccweb.com
Password for dons@pccweb.com:
Cloning 3 application files.
Deploying new version.
Checking if new version is ready to serve.
Will check again in 1 seconds.
Checking if new version is ready to serve.
Will check again in 2 seconds.
Checking if new version is ready to serve.
Closing update: new version is ready to start serving.
Uploading index definitions.

C:\Users\dons>
```

Now open a browser and go to "your application name".appspot.com/hello to run our test. If all is correct you'll receive the below confirmation.



Write

In this next example, we'll create a Table (called a Model in the DataStore) and store some test data (Entities). Open the YAML file created previously and add the "write" section:

```
application: sqlbold
version: 1
runtime: python
api_version: 1

handlers:
- url: /hello.*
  script: hello.py

- url: /write.*
  script: write.py
```

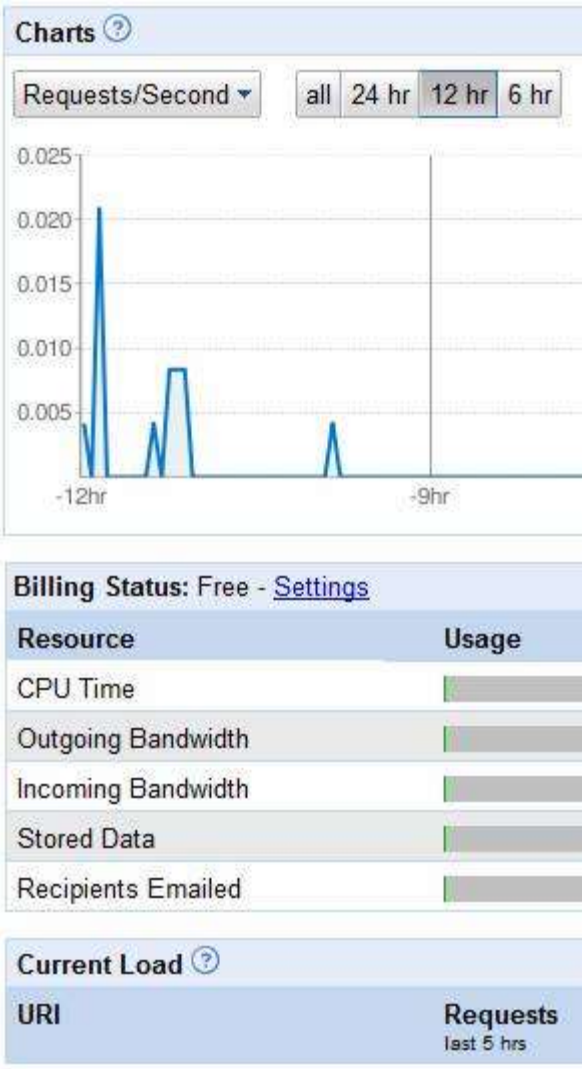
This will map any URL starting with the word "write" to a Python page we'll create called "write.py". Next, create the write.py page and locate it in the "helloworld" directory. Add the following code into write.py:

```
from google.appengine.ext import db

class Cars(db.Model):
    make = db.StringProperty()
    year = db.IntegerProperty()

myCar = Cars(make="Ford", year=2009)
myCar.put()
```

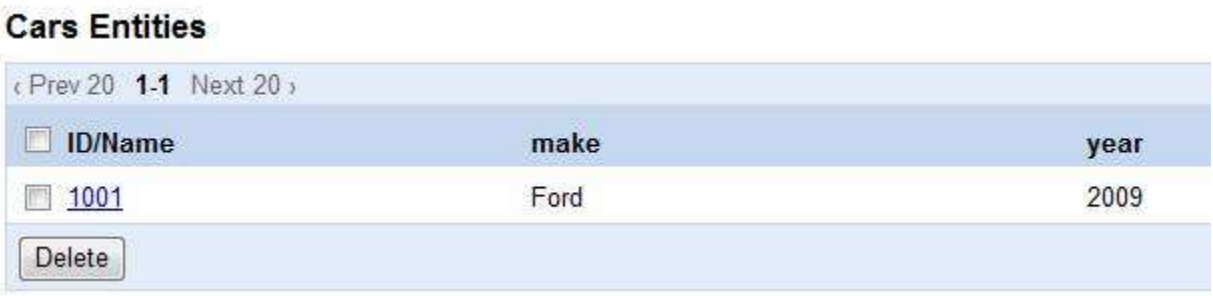
The first line, "import db", accesses the Google database API. Next, the class block creates a Table (called a Model in the DataStore) of two columns; a string called "make" and an Int called "year". The Model will be called "Cars". The "myCar" line creates one row of data. Lastly the "put()" is executed. Put is our method to "INSERT".

Administration Console

Let's enter the Google Administration Console to view the newly inserted data. The Console can be accessed from: http://appengine.google.com/. After logging in, the first screen will list our Applications (Databases). Entering an Application will bring up the Dashboard pictured below.



The Dashboard gives us a snapshot of our processes and resources being utilized. On the left of the chart is a link called "Data Viewer". The Data Viewer is the web interface to our data and shows the record just entered.



From this interface we can add, edit, and delete data as well as view. In addition, there is a window to Query. Click the "Query" radio button towards the top of the screen to open a query text area.

GQL Query

Querying data in the DataStore is done with a SQL like language called GQL. For example, this statement returns the newly created record:

SELECT * FROM Cars WHERE make = 'Ford'

The GQL is not a robust language like PL/SQL or TSQL, but is adequate for presenting data to forward facing web applications. There are WHERE, ORDER BY, and LIMIT keywords. The OR command doesn't exist for WHERE statements, but there is an IN grouping clause that can be used. The GQL reference is located at this
URL: http://code.google.com/appengine/docs/python/datastore/gqlreference.html .

Conclusion

The DataStore is a fast-distributed web service database located in the Google cloud. It's managed by APIs written in either Python or JAVA. There is a SQL like language for working with data called GCL.